Data Structures and Algorithms

Unit-1 Assignment Report

Name: Manas Aggarwal

Roll No: 2501730302

Section: A

Submitted to: Prof. Neetu Chauhan

1. ADT and Stack

- **What is an ADT?** ADT (Abstract Data Type) is a high-level description of data and operations (like push/pop for stack), without caring how it's implemented (array or list). Data Structure is the actual implementation, e.g., list for stack.
- **Why Stack Operations O(1)?** Push/pop access top element directly (append/pop in list), constant time regardless of size. Best/avg/worst all O(1). Used here to trace Hanoi moves by pushing each print statement.

2. Factorial & Fibonacci Analysis

- **Factorial:** Time O(n) (n calls, each O(1) work), Space O(n) (call stack depth n). Best/avg/worst same as linear. Base case n=0/1 stops recursion.
- **Fibonacci Naive:** Time O(2^n) (exponential calls, e.g., 2.6M for n=30), Space O(n). Slow for large n due to recomputing subproblems (fib(28) called many times).
- **Fib Memoized:** Time O(n) (each fib(i) computed once), Space O(n). Calls ~2n (e.g., 59 for n=30), proves efficiency. Memo stores results to avoid recompute.

3. Tower of Hanoi

- Time O(2^n - 1) (~7 moves for n=3), as each n calls 2*(n-1) +1. Space O(n) (max depth n). Base n=1: single move; subproblems: move n-1 to aux, move n to dest, move n-1 to dest.

4. Binary Search

- Time: Recurrence T(n) = T(n/2) + O(1) solves to O(log n). Best: O(1) (mid match), Avg/Worst: O(log n). Space O(log n) depth. Divide: split array at mid, conquer half. Handles empty (-1), not found (-1).

5. Paradigm Reflection

- **Binary Search: Divide & Conquer** Splits problem (array) into smaller halves, solves by recursing on one. Real-life: Merge sort divides list.

- **Fib Memoized: Dynamic Programming** Solves overlapping subproblems (fib(5) reused), memo avoids recompute. Solves inefficiency of naive recursion.
- **Tower of Hanoi: Recursion** Base: n=1 move. Subproblem: same for smaller n. Real-life Divide/Conquer: Google search divides queries; Greedy: coin change (pick largest first); DP: knapsack (build table of sub-solutions).